B6 Reproducibility

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B6 Reproducibility 3
B6-L1 Seeds and random generators 3
B6-L2 Record the data, config and code behind every result 7
B6-L3 Pinned dependencies and lock files 11
Track B best-practice checklist 15
Appendix: solutions and answer keys 16

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B6 Reproducibility

Roadmap: Prerequisite for all months. Seeds, config, data versioning and pinned dependencies.

B6-L1 Seeds and random generators

Concept

A result you cannot rerun is a result you cannot defend. Randomness enters analysis through sampling, train/test splits, bootstrap intervals, model initialisation and synthetic data. Control each source.

Create a generator and pass it in. NumPy 1.17 introduced `numpy.random.Generator`. Examples use NumPy 2.4.

```
rng = np.random.default_rng(20260925)      # at the edge, once
sample = draw_sample(ids, k=500, rng=rng)  # the function receives rng
```

- `np.random.default_rng(seed)` gives the same stream of numbers for the same seed, on every run and machine with the same NumPy version.
- Functions take `rng: np.random.Generator` as a parameter. They never call `np.random.seed()` or the legacy `np.random.rand()`, which share one hidden global state across your whole program and every library you import.
- One generator, used twice, gives two **different** draws. That is correct: two samples should differ. Rerunning the whole script gives the same pair.
- Need independent streams for parallel workers? `rng.spawn(n)` returns `n` child generators.

Libraries take the seed or generator as a parameter too: `train_test_split(..., random_state=42)` in scikit-learn, `df.sample(n, random_state=rng)` in pandas.

Two hidden sources of non-determinism:

- `hash()` **on strings changes between Python processes** (hash randomisation, `PYTHONHASHSEED`). `hash(respondent_id) % 10` puts the same respondent in a different bucket on each run. Use `hashlib.sha256` for stable buckets.
- **Set iteration order** for strings also changes between runs. Sort before you rely on order.

Record the seed in the run log with the results, so anyone reruns the exact analysis.

Why it matters

A holdout set was defined as `hash(respondent_id) % 10 == 0`. The model scored AUC 0.81 in the notebook. The scheduled retraining job, a new Python process, put a different 10% in the holdout. Some of the old test respondents were now in training, and the reported AUC rose to 0.84 with no change to the model. The team spent a week chasing an improvement that did not exist.

A bucket from `hashlib.sha256` of the ID is the same in every process.

Worked example: weak code

```
 1 import numpy as np
 2
 3 np.random.seed(1)
 4
 5
 6 def draw_sample(ids, k):
 7     return list(np.random.choice(ids, size=k, replace=False))
 8
 9
10 def holdout(ids):
11     return [i for i in ids if hash(i) % 10 == 0]
```

1. **Line 3.** Global state, set at import: any other call to np.random changes what draw_sample returns.
2. **Line 7.** Reads the hidden global generator.
3. **Line 11.** String hash() is randomised per process: a different holdout every run.

Worked example: strong code

```
 1 import hashlib
 2
 3 import numpy as np
 4
 5
 6 def draw_sample(ids: list[str], k: int, rng: np.random.Generator) ->
   list[str]:
 7     """Draw k distinct IDs using the generator passed in."""
 8     return [str(x) for x in rng.choice(ids, size=k, replace=False)]
 9
10
11 def bucket(respondent_id: str, n_buckets: int) -> int:
12     """Stable bucket for an ID, the same in every process."""
13     digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
14     return int(digest, 16) % n_buckets
15
16
17 def main(seed: int) -> None:
18     """The edge: create the generator once and log the seed."""
```

```
19      rng = np.random.default_rng(seed)
20      ...
```

1. **Line 6.** The randomness is a parameter: tests pass a seeded generator.
2. **Line 13.** A cryptographic hash is stable across processes and machines.
3. **Line 19.** One generator, created at the edge from a logged seed.

Seed once at the edge, pass the generator in, and never use hash() for anything that must be stable.

Recap card

- **Reproducible randomness in NumPy 2** rng = np.random.default_rng(seed), created once at the edge and passed in.
- **Why not np.random.seed?** It seeds one global generator that every library shares and advances.
- **Stable bucket for an ID** int(hashlib.sha256(id.encode()).hexdigest(), 16) % n. Never hash(), which is randomised per process.
- **Independent streams for workers** rng.spawn(n).

Practice

Exercise 1. Multiple choice (level 1)

Which is the recommended way to get reproducible random numbers in NumPy 2?

- a) `np.random.seed(42)` at the top of the script
- b) `rng = np.random.default_rng(42)`, passed to the functions that need it
- c) `random.seed(42)` from the standard library
- d) No seed. Results average out.

Why?

- r1) Each Generator owns its state, so no other code can change the stream it gives your function.
- r2) np.random.seed was removed in NumPy 2.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
1 import numpy as np
2
3 rng = np.random.default_rng(1)
4 first = rng.choice(10, size=3, replace=False).tolist()
5 second = rng.choice(10, size=3, replace=False).tolist()
6 print(first == second)
```

Exercise 3. Fill the gap (level 2)

Create a seeded generator and give four parallel workers independent streams.

```
rng = np.random._____(1)(20260925)
worker_rngs = rng._____(2)(4)
```

Exercise 4. Write the function (level 3)

Write `bucket(respondent_id: str, n_buckets: int) -> int`. It returns a number from 0 to `n_buckets - 1` that is the same for the same ID in every Python process. Use `hashlib.sha256` on the UTF-8 bytes of the ID, read the hex digest as an integer, and take it modulo `n_buckets`. Do not use `hash()`.

Your code must pass these checks:

- Buckets must come from int(sha256(id.encode('utf-8')).hexdigest(), 16) % n_buckets.
- Every bucket must be between 0 and n_buckets - 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Stable bucket for an ID, the same in every process.
def bucket(respondent_id: str, n_buckets: int) -> int: ...
```

Starter code:

```
def bucket(respondent_id, n_buckets):
    return hash(respondent_id) % n_buckets
```

Exercise 5. Spot the bug (level 4)

Two analysts run the same bootstrap with seed 7 and get different confidence intervals. One of them imports a plotting helper that draws random jitter with `np.random`. Tap the line that makes the result depend on other code, and fix the function.

What you need to know.

`np.asarray(values)` turns a Python list into a NumPy array, so array methods such as `.mean()` work on it. An input that is already an array passes through without a copy.

```
np.asarray([1.0, 2.0]).mean()   # 1.5
```

```
import numpy as np


def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
    list[float]:
    """Bootstrap means of values."""
```

```
    np.random.seed(seed)
    out = []
    for _ in range(n_boot):
        draw = np.random.choice(values, size=len(values), replace=True)
        out.append(float(draw.mean()))
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- np.random.default_rng(seed) returns a Generator; Generator.spawn(n) returns n independent child generators. (verified: NumPy random documentation; run on NumPy 2.4.6 in this repository.)
- Python's hash() of a str differs between processes unless PYTHONHASHSEED is fixed. (verified: Python data model documentation (object.**hash**); observed across two processes in this repository.)

B6-L2 Record the data, config and code behind every result

Concept

To rerun a result you need three things exactly as they were: the **data**, the **config** and the **code**. Record all three with every run, in a small **run manifest**.

```
manifest = {
    "run_id": run_id,
    "started_utc": datetime.now(timezone.utc).isoformat(),
    "input_sha256": file_sha256(input_path),
    "config_sha256": config_sha256(config),
    "git_commit": git_commit,
    "seed": seed,
    "packages": {"pandas": version("pandas"), "numpy": version("numpy")},
}
```

Data. A file name and a date do not identify content. Suppliers resend files under the same name. A **content hash** does: `hashlib.sha256` over the file bytes changes if any byte changes. `hashlib.file_digest(f, "sha256")` (Python 3.11 and later) reads the file in chunks, so large files do not load into memory. Keep raw inputs immutable: write each delivery to a new path and never overwrite it. For lakehouse tables, record the Delta table version instead (lesson A3-L3).

Config. Hash the config you loaded, not the file on disk, so defaults count. Serialise with `json.dumps(config, sort_keys=True)` first. Without `sort_keys`, the same settings in a different key order give a different hash.

Code. Record the Git commit. Refuse to produce client output from a working tree with uncommitted changes, or record that it was dirty.

Environment. `importlib.metadata.version("pandas")` returns the installed version. The lock file (lesson B6-L3) covers the rest.

Store the manifest next to the output, or as a row in a run log table. When someone asks "why did this number change?", compare two manifests. The field that differs is where to look.

Why it matters

A client queried a brand share that moved 3 points between two versions of the same wave's report. The analysis folder held `wave12.csv`, the config and the notebook, all edited since. Nobody knew whether the supplier had resent the file, a setting had changed, or the code had. Reconstructing the answer took four days.

With manifests, the two runs differed in one field: `input_sha256`. The supplier had resent the file with 212 late completes.

Worked example: weak code

```
1 df = pd.read_csv("data/wave12.csv")
2 results = analyse(df, CONFIG)
2 results.to_csv(f"out/wave12_{date.today()}.csv")
```

1. **Line 1.** The name says nothing about the content. A resent file has the same name.
2. **Line 3.** A date, but no record of the input, config, code or seed behind it.

Worked example: strong code

```
1  import hashlib
2  import json
3  from pathlib import Path
4
5
6  def file_sha256(path: Path) -> str:
7      """SHA-256 of a file's bytes, read in chunks."""
8      with path.open("rb") as f:
1          return hashlib.file_digest(f, "sha256").hexdigest()
10
11
12 def config_sha256(config: dict[str, object]) -> str:
13     """SHA-256 of a config, independent of key order."""
```

```
    text = json.dumps(config, sort_keys=True)
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

1. **Line 9.** Content identity: any changed byte changes the hash.
2. **Line 14.** Stable serialisation: key order does not change the hash.

Hash the input and the loaded config, record the commit and the seed, and store the manifest with the output.

Recap card

- **Three things to record for a rerun** Data (content hash or table version), config (hash of the loaded config), code (Git commit).
- **Hash a large file** with path.open("rb") as f: hashlib.file_digest(f, "sha256") (Python 3.11+).
- **Stable config hash** sha256 of json.dumps(config, sort_keys=True).encode("utf-8").
- **Raw inputs** Immutable: each delivery at a new path, never overwritten.

Practice

Exercise 1. Multiple choice (level 1)

Which value best identifies the exact input file a result came from?

- a) The file name
- b) The file's modified time
- c) The SHA-256 of the file's bytes
- d) The row count

Why?

- r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.
- r2) Hashes are shorter than file names.

Exercise 2. Predict the output (level 2)

What does this print?

```
import json

print(json.dumps({"min_base": 50, "cap": 5.0}, sort_keys=True))
```

Exercise 3. Fill the gap (level 2)

Hash a large file without loading it into memory.

```
with path.open("_____(1)") as f:
```

```
    digest = hashlib._____(2)(f, "sha256").hexdigest()
```

Exercise 4. Write the function (level 3)

Write two functions.

- `file_sha256(path: Path) -> str` returns the hex SHA-256 of the file's bytes.
- `config_sha256(config: dict[str, object]) -> str` returns the hex SHA-256 of `json.dumps(config, sort_keys=True)` encoded as UTF-8, so key order does not matter.

Your code must pass these checks:

- file_sha256 must hash the file's raw bytes with SHA-256.
- Key order must not change the config hash. Use json.dumps(..., sort_keys=True).
- Hash json.dumps(config, sort_keys=True) encoded as UTF-8.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# SHA-256 of a file's bytes, read in chunks.
def file_sha256(path: Path) -> str: ...

# SHA-256 of a config, independent of key order.
def config_sha256(config: dict[str, object]) -> str: ...
```

Starter code:

```python
import hashlib
import json
from pathlib import Path


def file_sha256(path):
    ...


def config_sha256(config):
    return hashlib.sha256(str(config).encode()).hexdigest()
```

Exercise 5. Architecture design (level 4)

Choose what the run manifest records for a monthly client report built from a supplier CSV.

Options: SHA-256 of the input file, with the raw file kept unchanged, The input file name.

1. Input data
2. Settings

3. Code
4. Where the manifest lives

Sources

- hashlib.file_digest was added in Python 3.11. (verified: Python hashlib documentation; run on Python 3.11 in this repository.)

B6-L3 Pinned dependencies and lock files

Concept

`pyproject.toml` states what your project **accepts**: `pandas>=3.0,<4`. A **lock file** records what was **installed**: `pandas 3.0.6`, `numpy 2.4.6`, and every indirect dependency, each with a file hash. You need both.

- The ranges in `pyproject.toml` say which versions your code supports. Keep them honest: a lower bound you have tested, an upper bound at the next major version when the library breaks things across majors.
- The lock file makes every install identical: your laptop, a colleague's, CI and the scheduled job. Commit it.

With **uv** (examples use uv 0.8):

```
1 uv lock                  # resolve pyproject.toml ranges, write uv.lock
2 uv sync --locked         # install exactly uv.lock; fail if it is out of
     date
3 uv lock --upgrade-package pandas   # move one package on purpose
```

`uv sync --locked` exits with an error when `pyproject.toml` has changed since the lock was written: "The lockfile at `uv.lock` needs to be updated, but `--locked` was provided." Use it in CI, so a dependency change without a new lock fails the build instead of resolving something new on the runner.

Without uv, `pip-compile --generate-hashes` (pip-tools) writes a fully pinned `requirements.txt` with hashes, and `pip install --require-hashes -r requirements.txt` refuses any file whose hash differs.

`pip freeze > requirements.txt` is a weaker substitute. It lists what happens to be in your environment, including packages you installed by hand for a one-off test, with no hashes and no record of why each is there.

Upgrade on purpose: one dependency at a time, on a branch, with tests. Record the reason in the commit message. Never let a fresh install pick up new versions silently.

Why it matters

A forecasting job ran on a fresh container every night. `requirements.txt` said `pandas>=2.0`. The night pandas 3.0 was released, the container installed it. The job's text columns switched to the new `str` dtype, a `.str.contains` call behaved differently on missing values, and the job filtered out 6% of rows. Nobody had changed a line of code.

A committed lock file, installed with `uv sync --locked`, keeps the old pandas until someone upgrades it deliberately, with tests.

Worked example: weak code

```
  1 # requirements.txt
(1) pandas
(2) scikit-learn>=1.0
  4 numpy
(3) jupyter
```

1. **Line 2.** No bounds: any future major version installs on the next fresh build.
2. **Line 3.** A lower bound only. Every build resolves whatever is newest that day.
3. **Line 5.** A development tool mixed into the runtime requirements.

Worked example: strong code

```
  1 # pyproject.toml
  2 [project]
  3 requires-python = ">=3.12"
(1) dependencies = ["pandas>=3.0,<4", "scikit-learn>=1.8,<2", "numpy>=2.4,<3"]
  5
  6 [dependency-groups]
(2) dev = ["pytest>=9", "ruff>=0.13", "jupyterlab>=4"]
  8
  9 # CI
(3) uv sync --locked --group dev
```

1. **Line 4.** Honest ranges: tested lower bounds, next major excluded.
2. **Line 7.** Development tools in their own group, not in the runtime install.
3. **Line 10.** Installs exactly uv.lock. A stale lock fails the build.

Ranges in pyproject.toml, exact versions and hashes in a committed lock file, and --locked in CI.

Recap card

- **pyproject.toml vs lock file** pyproject.toml: ranges you support. Lock: exact versions and hashes of everything installed.
- **uv sync --locked** Installs exactly uv.lock and fails if pyproject.toml changed without a new lock.

- **Hashes without uv** pip-compile --generate-hashes, then pip install --require-hashes -r requirements.txt.
- **Why not pip freeze?** It captures one environment's accidents, with no hashes and no reasons.

Practice

Exercise 1. Multiple choice (level 1)

What does a lock file add that `pandas>=3.0,<4` in pyproject.toml does not?

- a) The exact version of every direct and indirect dependency, with file hashes.
- b) A list of packages your code imports.
- c) Faster imports at runtime.
- d) Nothing. They hold the same information.

Why?

- r1) Every install then gets identical packages, on every machine.
- r2) It lets pip skip dependency resolution entirely on first install.

Exercise 2. Predict the output (level 2)

You add `polars>=1.30` to `dependencies` in pyproject.toml and push without running `uv lock`. CI runs `uv sync --locked`. What happens?

```
1 $ uv sync --locked
```

Exercise 3. Fill the gap (level 2)

Write a pinned requirements file with hashes using pip-tools, then install it so any mismatched file is refused.

```
1 pip-compile --_____(1) requirements.in
2 python -m pip install --_____(2) -r requirements.txt
```

Exercise 4. Put in order (level 3)

Order the steps to upgrade scikit-learn in a locked project.

- Commit pyproject.toml and uv.lock with the reason
- Create a branch
- uv lock --upgrade-package scikit-learn
- Raise the bound in pyproject.toml if the new version is outside the range
- Run the test suite and compare key outputs with the last release

Exercise 5. Architecture design (level 4)

Set up dependency management for a nightly scoring job.

Options: Tested lower bounds, next major excluded, in pyproject.toml, Package names only, no versions, Exact == pins in pyproject.toml, no lock.

1. Declared dependencies
2. Exact versions
3. Install in CI and the job

Sources

- uv sync --locked fails when uv.lock needs updating, with the message quoted in this lesson. (verified: Run with uv 0.8.17 in this repository's scratch directory.)
- pip-compile --generate-hashes and pip install --require-hashes. (search-only: pip-tools and pip documentation (not run here).)
- Series.str.contains returns None for a missing value on pandas 2.3 (object dtype) and False on pandas 3.0 (str dtype). (verified: Run on pandas 2.3.3 and 3.0.6 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B6-L1 Seeds and random generators

B6-L1 exercise 1

Hints. (1) Which option has no shared global state? (2) Which one can a test control? (3) default_rng.

Answer. b) `rng = np.random.default_rng(42)`, passed to the functions that need it Because: r1) Each Generator owns its state, so no other code can change the stream it gives your function.

`default_rng(seed)` gives an independent, seeded generator. Pass it in so each function's randomness is explicit.

Common wrong answers.

- `a`: This seeds a global generator that every library shares. Any extra call anywhere shifts your numbers.
- `c`: The standard library seed does not affect NumPy's generators.
- `d`: Unseeded runs cannot be reproduced or debugged.
- `b|r2`: Right answer, wrong reason. np.random.seed still exists as legacy API. Its global state is the problem.

B6-L1 exercise 2

Hints. (1) Does the generator's state change after a draw? (2) Is the second draw from the same position? (3) It continues the stream.

Answer.

```
1 False
```

The generator advances. Two draws differ, and rerunning the script repeats both.

Common wrong answers.

- `True`: One generator moves on after each draw. Only a fresh generator with the same seed repeats the first draw.

B6-L1 exercise 3

Hints. (1) The function that returns a Generator. (2) The Generator method that makes child generators. (3) `default_rng` and `spawn`.

Answer.

```
rng = np.random.default_rng(20260925)
worker_rngs = rng.spawn(4)
```

`default_rng` builds the parent. `spawn(4)` returns four independent children.

Common wrong answers.

- g1 = `seed`: np.random.seed sets the global generator and returns None.

B6-L1 exercise 4

Hints. (1) `hashlib.sha256(data)` needs bytes: `respondent_id.encode("utf-8")`. (2) `.hexdigest()` returns hex text. `int(text, 16)` reads it. (3) Finish with `% n_buckets`.

Answer.

```
import hashlib


def bucket(respondent_id: str, n_buckets: int) -> int:
    """Stable bucket for an ID, the same in every process.

    Args:
        respondent_id: The ID to bucket.
        n_buckets: Number of buckets.

    Returns:
        An integer from 0 to n_buckets - 1.
    """
    digest = hashlib.sha256(respondent_id.encode("utf-8")).hexdigest()
    return int(digest, 16) % n_buckets
```

SHA-256 of the ID's bytes is the same everywhere, so the bucket is too. `hash()` is salted per process for strings.

B6-L1 exercise 5

Hints. (1) Which generator does np.random.choice use? (2) Who else can move that generator? (3) Use a local default_rng(seed).

Answer. The bug is on line 6, 9.

Fixed version:

```
import numpy as np


```

```
4 def bootstrap_means(values: list[float], n_boot: int, seed: int) ->
    list[float]:
5     """Bootstrap means of values, reproducible from seed."""
6     rng = np.random.default_rng(seed)
7     data = np.asarray(values)
8     return [float(rng.choice(data, size=len(data), replace=True).mean())
    for _ in range(n_boot)]
```

`np.random.seed` and `np.random.choice` use one global generator that any imported code advances. A local `default_rng(seed)` belongs to this function alone.

Common wrong answers.

- Line 10: Taking the mean is fine. The draws come from a shared global generator.

B6-L2 Record the data, config and code behind every result

B6-L2 exercise 1

Hints. (1) Which option changes when one value in the file changes? (2) Which stays the same when the file is copied? (3) A content hash.

Answer. c) The SHA-256 of the file's bytes Because: r1) Any change to the content changes the hash, and identical content gives the same hash everywhere.

A content hash is tied to the bytes, not to the name or the file system.

Common wrong answers.

- `a`: Suppliers resend changed files under the same name.
- `b`: Copying a file changes its modified time without changing content, and edits can keep it.
- `d`: A file with changed values and the same number of rows has the same count.
- `c|r2`: Right answer, wrong reason. Length does not matter. Content identity does.

B6-L2 exercise 2

Hints. (1) What does sort_keys do? (2) Which comes first alphabetically? (3) c before m.

Answer.

```
1 {"cap": 5.0, "min_base": 50}
```

Sorted keys make the text, and so the hash, independent of the order you wrote the config in.

Common wrong answers.

- `{"min_base": 50, "cap": 5.0}`: sort_keys=True orders keys alphabetically: cap before min_base.

B6-L2 exercise 3

Hints. (1) Hashes work on bytes. (2) Python 3.11 added a hashlib helper for file objects. (3) `rb` and `file_digest`.

Answer.

```
with path.open("rb") as f:
    digest = hashlib.file_digest(f, "sha256").hexdigest()
```

Binary mode gives the exact bytes. `hashlib.file_digest` reads them in chunks.

Common wrong answers.

- g1 = `r`: Text mode decodes and may change line endings. Hash the raw bytes with rb.

B6-L2 exercise 4

Hints. (1) Open the file in binary mode. (2) `hashlib.file_digest(f, "sha256")` or `hashlib.sha256(path.read_bytes())`. (3) For the config, `json.dumps(config, sort_keys=True).encode("utf-8")`.

Answer.

```
import hashlib
import json
from pathlib import Path


def file_sha256(path: Path) -> str:
    """SHA-256 of a file's bytes, read in chunks.

    Args:
        path: File to hash.

    Returns:
        The hex digest.
    """
    with path.open("rb") as f:
        return hashlib.file_digest(f, "sha256").hexdigest()


def config_sha256(config: dict[str, object]) -> str:
```

```
    """SHA-256 of a config, independent of key order.

    Args:
        config: A JSON-serialisable mapping.

    Returns:
        The hex digest of the sorted JSON text.
    """
    text = json.dumps(config, sort_keys=True)
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

Hash content, not names, and serialise configs in a fixed key order before hashing.

B6-L2 exercise 5

Hints. (1) What changes without changing its name? (2) What would you compare between two runs? (3) Store it where the next analyst will look.

Answer.

- Content hash of an immutable input.
- Hash of the loaded config, so defaults and key order are handled.
- Git commit plus a dirty flag.
- Stored with the output and in the run log, where comparisons are easy.

Record identities, not names: content hashes and commits. Keep them where a diff between runs is one query away.

Common wrong answers.

- data = `name`: A resent file keeps its name. Only a content hash shows the data changed.
- config = `file_path`: The file at that path changes later, and defaults are not in it.
- code = `notebook`: The notebook changes under the same name. The commit pins the code.
- store = `chat`: Chat is hard to search and compare. Keep manifests with the output.

B6-L3 Pinned dependencies and lock files

B6-L3 exercise 1

Hints. (1) How many versions does a range accept? (2) What about packages you did not list, such as numpy under pandas? (3) Exact versions plus hashes.

Answer. a) The exact version of every direct and indirect dependency, with file hashes. Because: r1) Every install then gets identical packages, on every machine.

Ranges declare what you support. The lock pins what you run, down to every indirect dependency.

Common wrong answers.

- `b`: Neither file is generated from imports. The lock records what the resolver chose.
- `c`: Locks affect installation, not import speed.
- `d`: A range accepts many versions. The lock records exactly one of each, including indirect dependencies.
- `a|r2`: Right answer, wrong reason. The benefit is identical installs, not skipped work.

B6-L3 exercise 2

Hints. (1) Does uv.lock include polars? (2) What does --locked assert? (3) That the lock would stay unchanged.

Answer.

```
1 The lockfile at `uv.lock` needs to be updated, but `--locked` was
      provided. To update the lockfile, run `uv lock`.
```

`--locked` asserts the lock matches pyproject.toml. It does not, so the command exits with an error and tells you to run `uv lock`.

Common wrong answers.

- `Installed polars`: --locked forbids changing the lock. The build fails instead of resolving on the runner.

B6-L3 exercise 3

Hints. (1) pip-compile has a flag that adds sha256 lines. (2) pip install has a flag that enforces them. (3) `generate-hashes` and `require-hashes`.

Answer.

```
1 pip-compile --generate-hashes requirements.in
2 python -m pip install --require-hashes -r requirements.txt
```

Hashes turn a pin into a guarantee: a different file with the same version number is rejected.

Common wrong answers.

- g1 = `upgrade`: --upgrade moves pins to newer versions. The task asks for hashes.

B6-L3 exercise 4

Hints. (1) The lock respects the range in pyproject.toml. (2) Tests come before the commit. (3) Branch, bound, lock, test, commit.

Answer.

1. Create a branch
2. Raise the bound in pyproject.toml if the new version is outside the range
3. uv lock --upgrade-package scikit-learn
4. Run the test suite and compare key outputs with the last release
5. Commit pyproject.toml and uv.lock with the reason

Widen the range, re-lock one package, prove it with tests, then commit both files together.

Common wrong answers.

- Locking before raising the bound resolves within the old range, so the upgrade you want is not allowed yet.
- Test before you commit, so a broken upgrade never lands.

B6-L3 exercise 5

Hints. (1) Which file says what you support, and which says what you run? (2) Where do indirect dependencies get pinned? (3) What makes CI fail on a stale lock?

Answer.

- Ranges in pyproject.toml say what the code supports.
- The committed lock pins every package with hashes.
- uv sync --locked installs exactly that, and fails on a stale lock.

Ranges for intent, a lock for fact, and a locked install everywhere the code runs.

Common wrong answers.

- declare = `none`: No bounds means any future major version is acceptable, which is rarely true.
- declare = `exact`: Exact pins in pyproject.toml miss indirect dependencies and block security fixes. Pin exactly in the lock.

- lock = `freeze`: freeze captures whatever is on one laptop, one-off installs included, with no hashes.
- install = `fresh`: Fresh resolution picks up new releases silently. That is how a job breaks with no code change.